

A decorative graphic consisting of a thin blue circle on the left side. A horizontal bar, divided into a solid blue left half and a light blue right half, extends from the circle across the top of the slide. Large blue square brackets are positioned on the left and right ends of this bar, framing the title text.

Stabilization and Synchronization of Logical Clocks

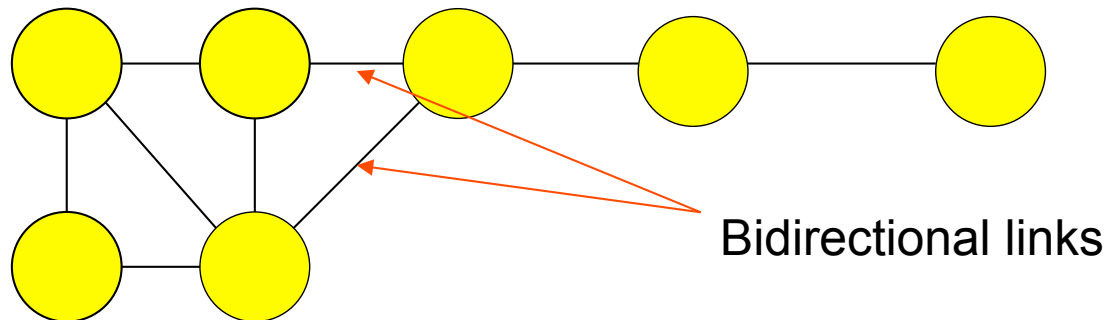
Franck Petit

UPMC

Paris

Preliminaries

- $G=(V,E)$ is a connected network
- V : set of n nodes/processes
- E : set of m bidirectional links
- N_p : set of neighboring nodes of p
- Memory shared between neighboring nodes



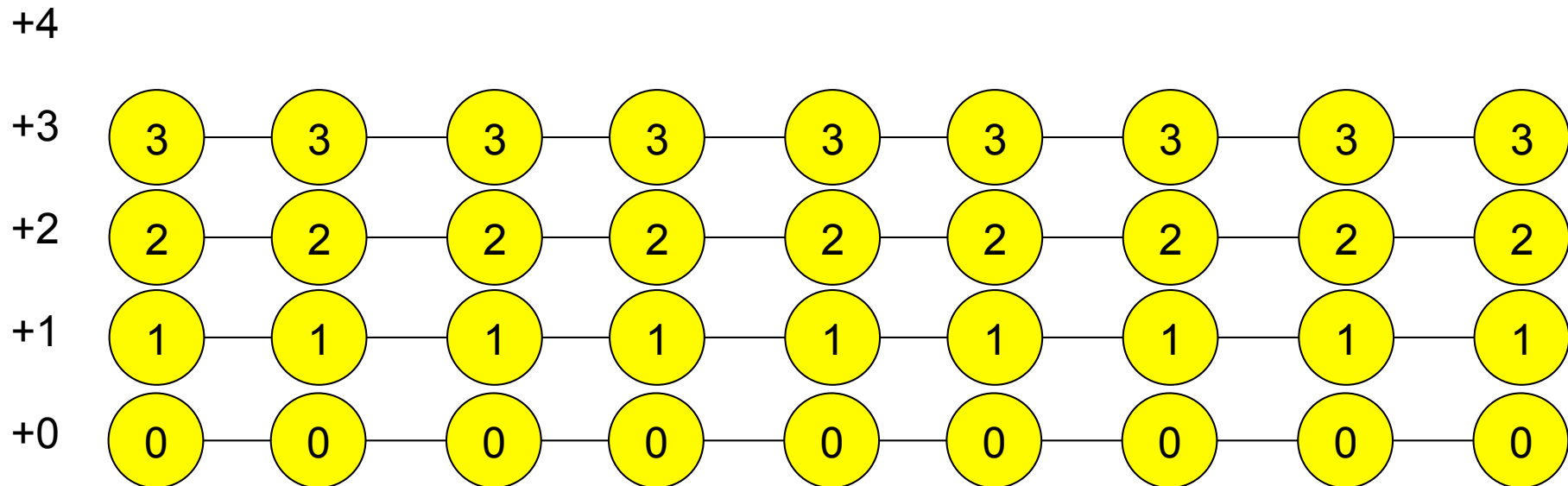
Preliminaries, Distributed Algorithm

- In each computation step:
 - According to its variables and the variables of its neighbors, a node/process is either enabled to execute an action or not
 - **Synchronous system**
 - **Every** enabled nodes execute an action atomically
 - **Asynchronous system**
 - **Some** enabled nodes are chosen by an unfair adversary
 - The chosen nodes execute an action atomically

Logical Clock Synchronization

- Each node p maintains a logical clock register r_p
- Synchronous/Asynchronous Environment

$$r_p := r_p + 1$$

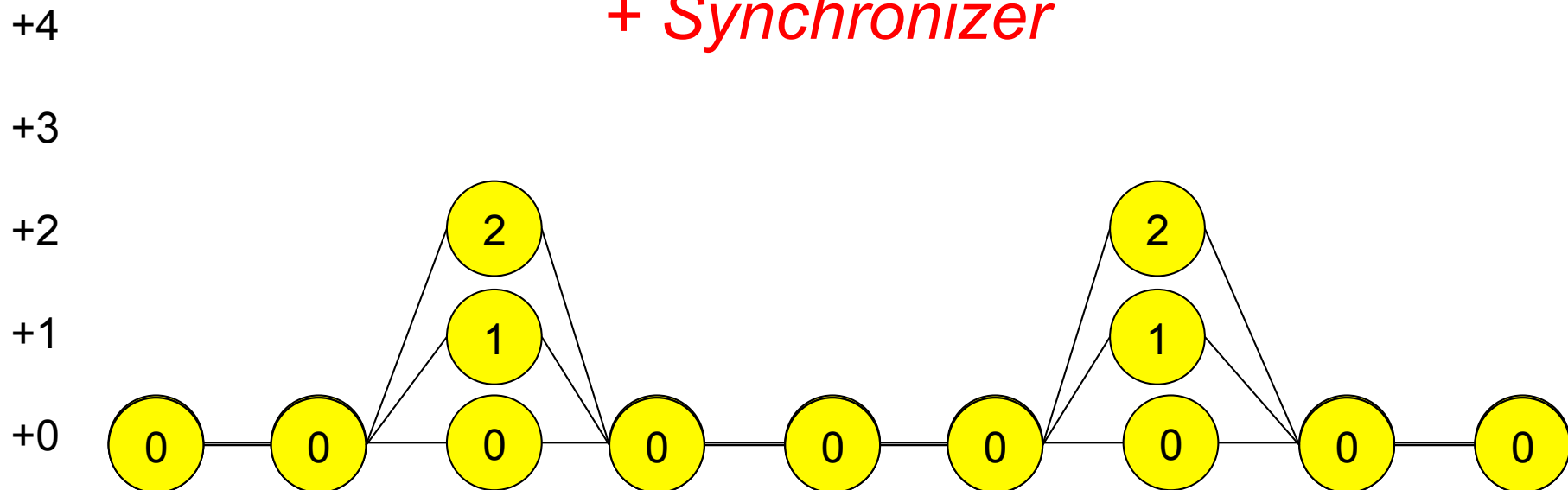


Logical Clock Synchronization

- Each node p maintains a logical clock register r_p
- Synchronous/Asynchronous Environment

$$r_p := r_p + 1$$

+ *Synchronizer*

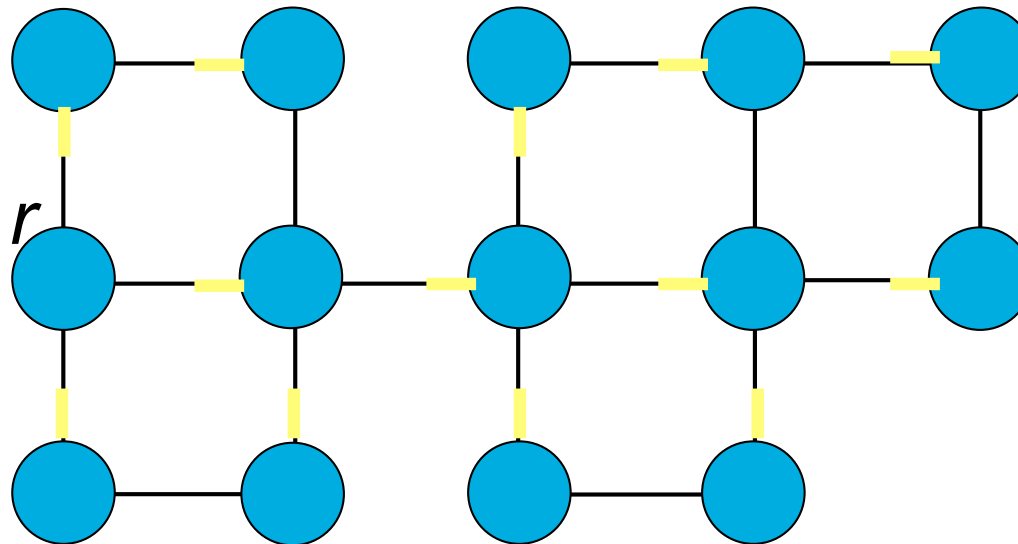


Global Synchronizer (asynchronous) distributed systems

In systems with **unique IDs** or a **particular node** (root, leader, main server...)?

- **Wave Algorithms, e.g.,**
 - Propagation of Information with Feedback (PIF)
 - Depth-First Token Circulation
- **Global Synchronization, e.g.,**
 - (Group) Mutual Exclusion
 - Leader Election
 - Reset
 - Logical Clock Synchronization
 - Rooted Spanning Tree
 - ...

Propagation of Information with Feedback



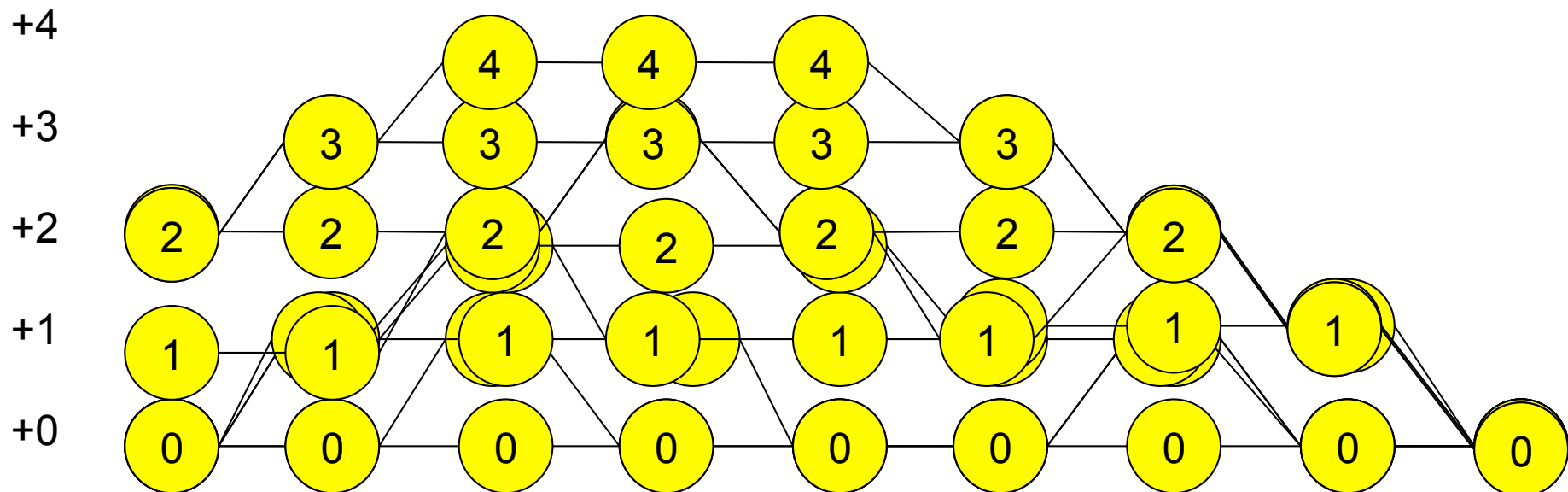
- For One Pulse:
 - Best Case: $O(D)$
 - Worst Case: $O(N)$

Can we provide better complexities?

Logical Clock Synchronization

- Each node p maintains a phase clock register r_p
- Synchronous/Asynchronous Environment

If $\forall q \in N_p : r_p \leq r_q$ then $r_p := r_p + 1$



Logical Clock Synchronization

Unison

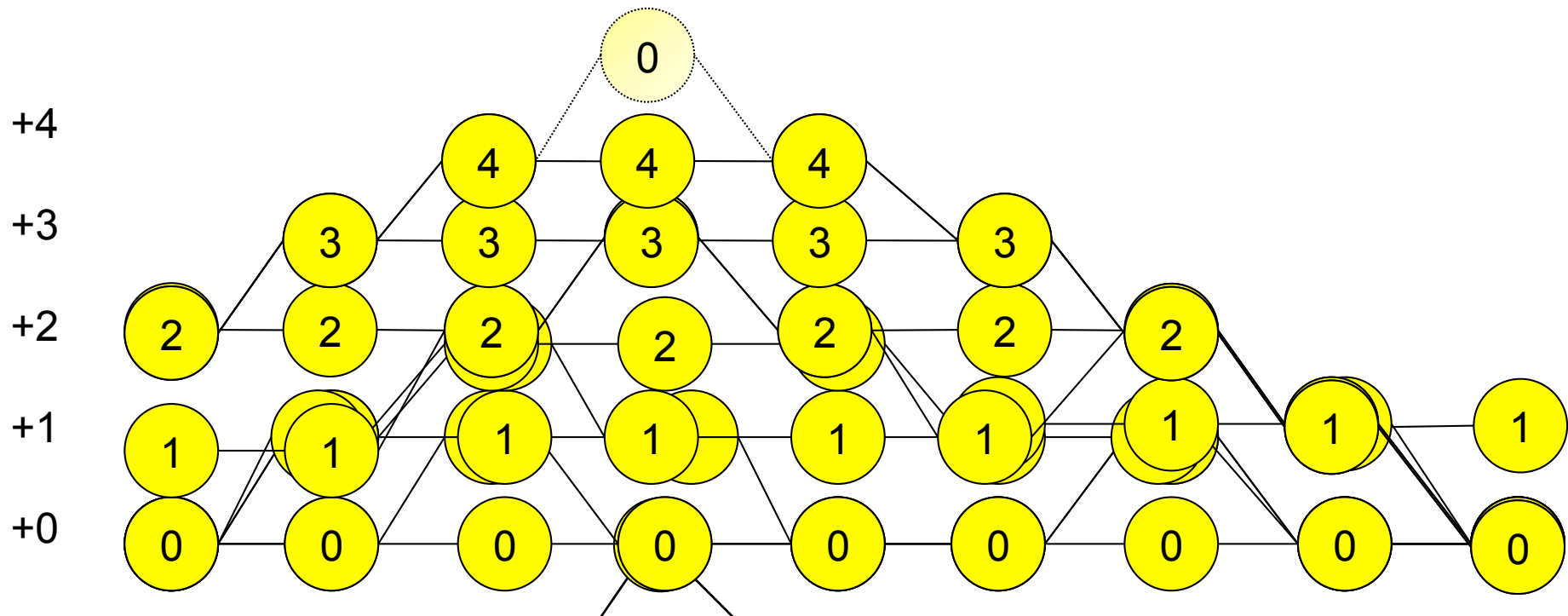
- Each node maintains a phase clock register r in $[0, \dots, K-1] = \mathbb{Z}_K$
- **Safety:** *The gap between the phase clock of two neighboring nodes is at most equal to $1 \pmod K$.*
- **No starvation (Vivacity):** *Each phase clock r is incremented by $1 \pmod K$ infinitely often.*

Logical Clock Synchronization

Unison

$K=5$

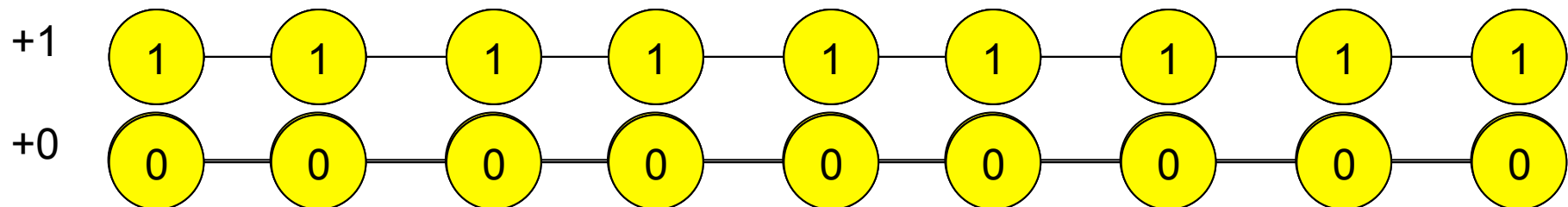
Minimality of K ?



Logical Clock Synchronization

- $K=2?$
- No possible order among the clocks

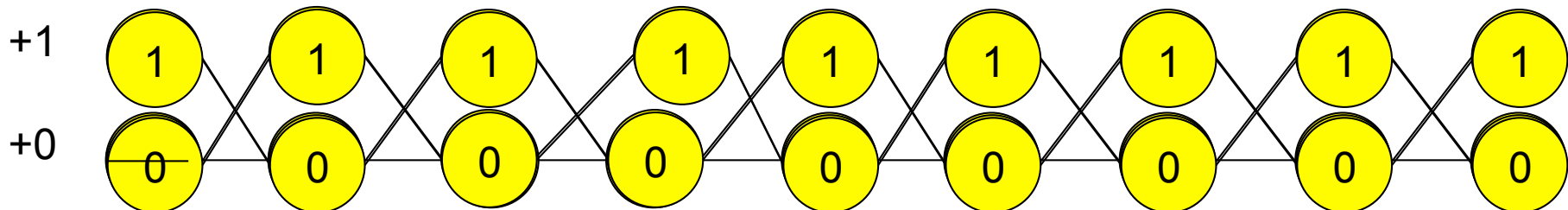
Synchronous



Logical Clock Synchronization

- $K=2$?
- No possible order among the clocks

Asynchronous



Logical Clock Synchronization

- Successor Function and Predecessor Function possible if $K \geq 3$
- $K=3$
 - Local total order $\leq_l$ over $Z_3 = \{0, 1, 2\}$

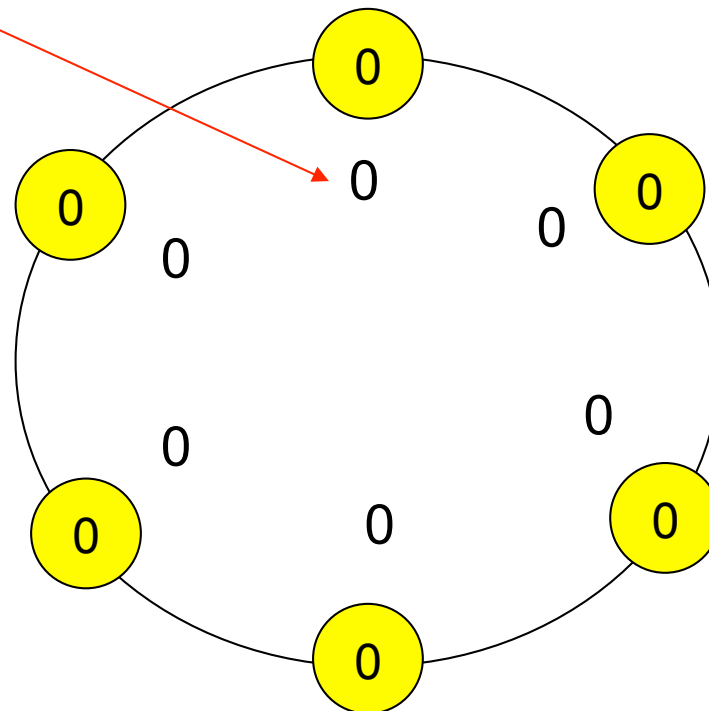
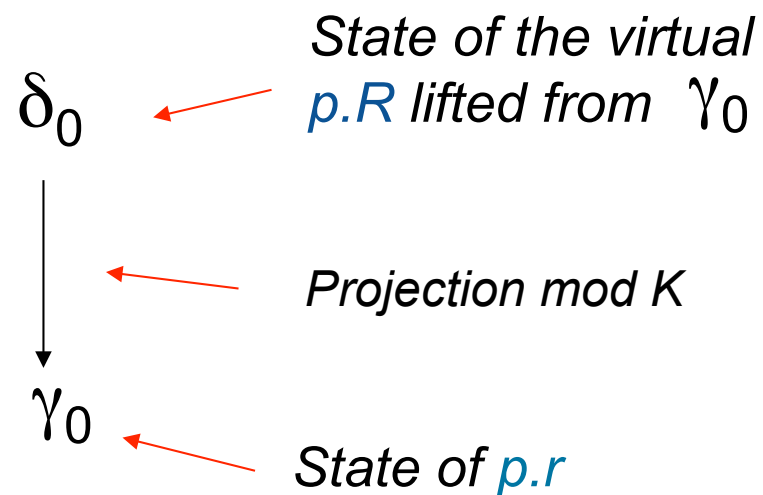
$$a \leq_l b \text{ iff } 0 \leq b-a \text{ mod } 3 \leq 1$$

$$0 \leq_l 1 ; 1 \leq_l 2 ; 2 \leq_l 0 ;$$

Logical Clock Synchronization

- $K=3$, **Lifting**

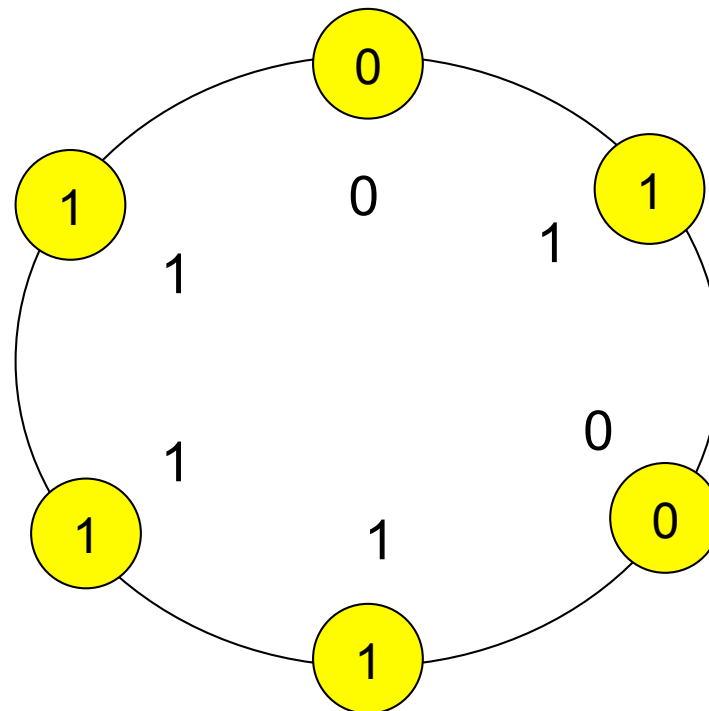
$p.R$, virtual register, it counts the number of increments of p



Logical Clock Synchronization

- $K=3$, Lifting

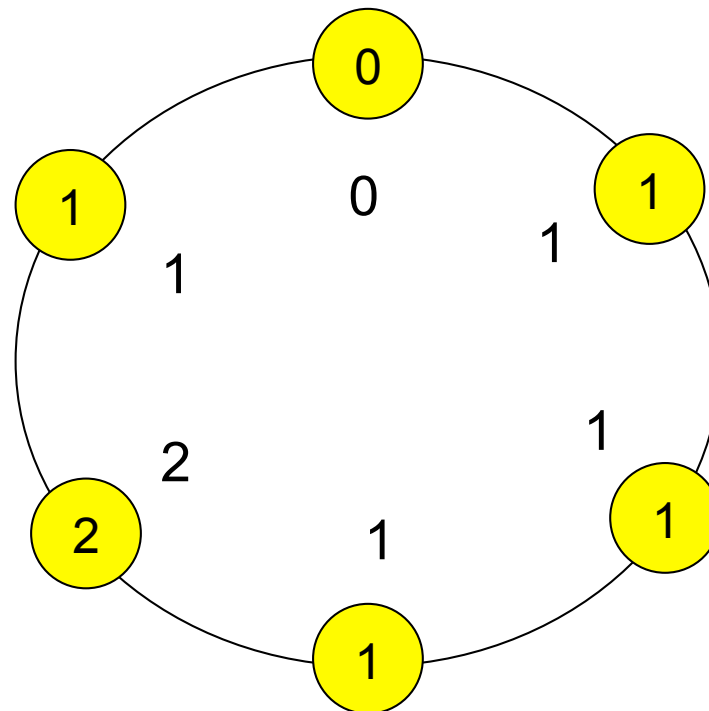
$$\begin{array}{ccc} \delta_0 & \xrightarrow{D_0} & \delta_1 \\ \downarrow & & \downarrow \\ \gamma_0 & \xrightarrow{D_0} & \gamma_1 \end{array}$$



Logical Clock Synchronization

- $K=3$, Lifting

$$\begin{array}{ccccc} \delta_0 & \xrightarrow{D_0} & \delta_1 & \xrightarrow{D_1} & \delta_2 \\ \downarrow & & \downarrow & & \downarrow \\ \gamma_0 & \xrightarrow{D_0} & \gamma_1 & \xrightarrow{D_1} & \gamma_2 \end{array}$$

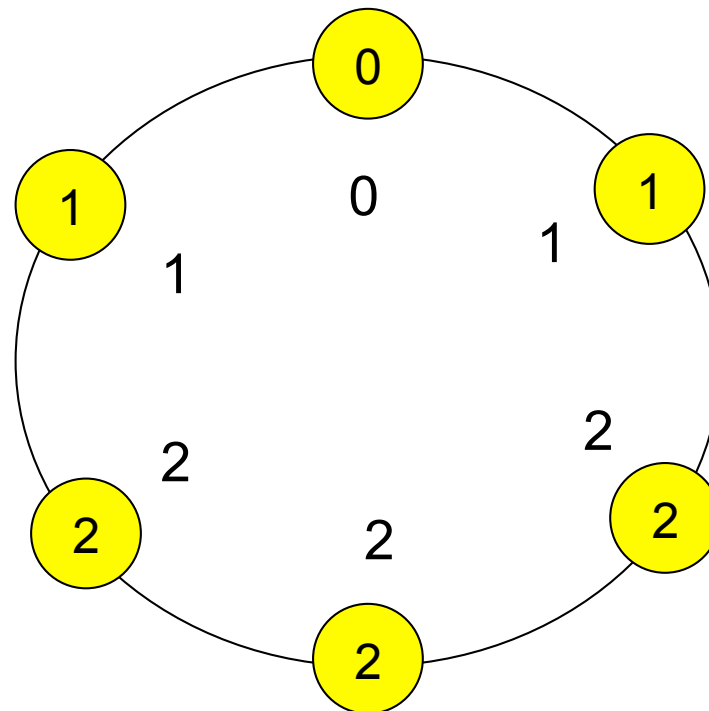


Logical Clock Synchronization

- K=3, Lifting

$$\delta_0 \xrightarrow{D_0} \delta_1 \xrightarrow{D_1} \delta_2 \xrightarrow{D_2} \dots$$

$$\begin{array}{c} \downarrow \quad \downarrow \quad \downarrow \\ \gamma_0 \xrightarrow{D_0} \gamma_1 \xrightarrow{D_1} \gamma_2 \xrightarrow{D_2} \dots \end{array}$$

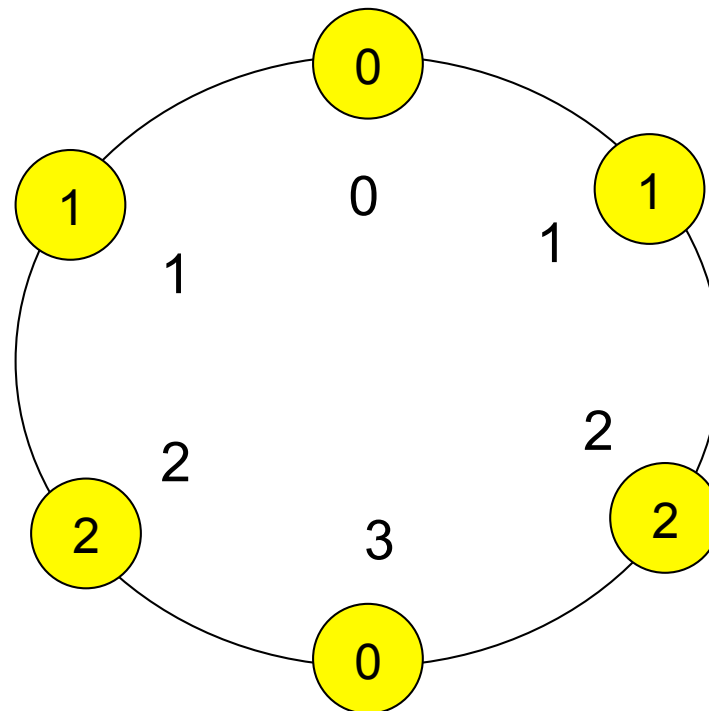


Logical Clock Synchronization

- $K=3$, Lifting

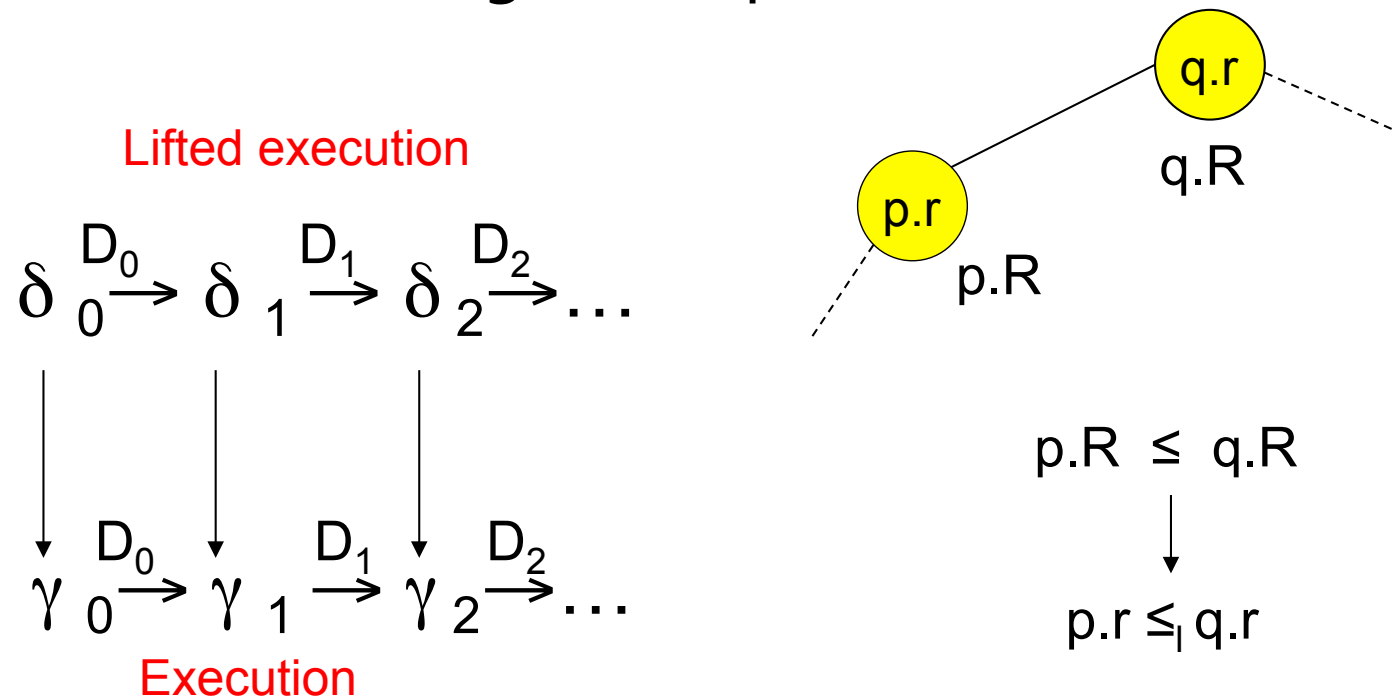
$$\delta_0 \xrightarrow{D_0} \delta_1 \xrightarrow{D_1} \delta_2 \xrightarrow{D_2} \dots$$

$$\begin{array}{c} \downarrow \quad \downarrow \quad \downarrow \\ \gamma_0 \xrightarrow{D_0} \gamma_1 \xrightarrow{D_1} \gamma_2 \xrightarrow{D_2} \dots \end{array}$$



Logical Clock Synchronization

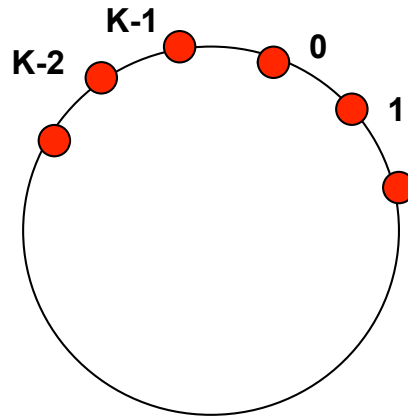
The lifting is compatible with the local ordering of Z_K



The lifting defines a global preorder over the nodes

Generalization over Z_K

- (Local) total order $\leq_l$ over $Z_K = \{0, 1, 2, \dots, K\}$
- Let $M \geq 1$ and $K \geq 2M+1$



$$a \leq_l b \text{ iff } 0 \leq b-a \bmod K \leq M$$

With $M=2$ and $K=5$, then for 1
 $4 \leq_l 1$; $0 \leq_l 1$; $1 \leq_l 1$; $1 \leq_l 2$; $1 \leq_l 3$;

Complexities

- Space : $O(1)$
- Time, for one pulse:
 - Best Case: $O(1)$
 - Worst Case: $O(D) \leftarrow [\text{DELAY}]$

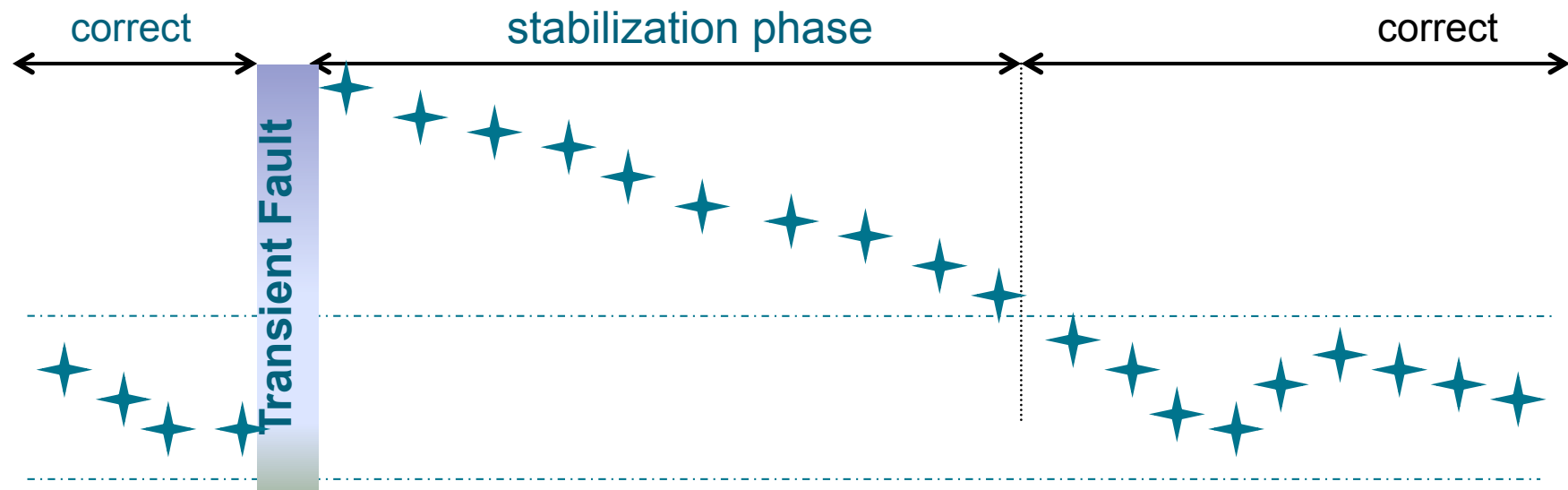
Fault Tolerance

Starting from an arbitrary configuration ?

Self-stabilization

A self-stabilizing system, regardless of its initial state, is guaranteed to converge to the intended behavior in finite time. [Dijkstra 74]

→ Transient Faults

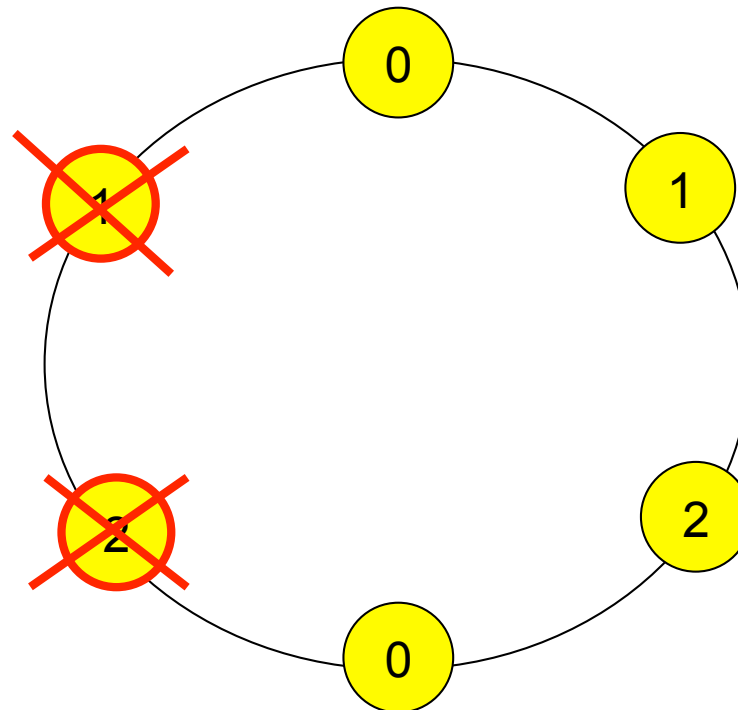


After a fault or an arbitrary initialization

○ $K=3$

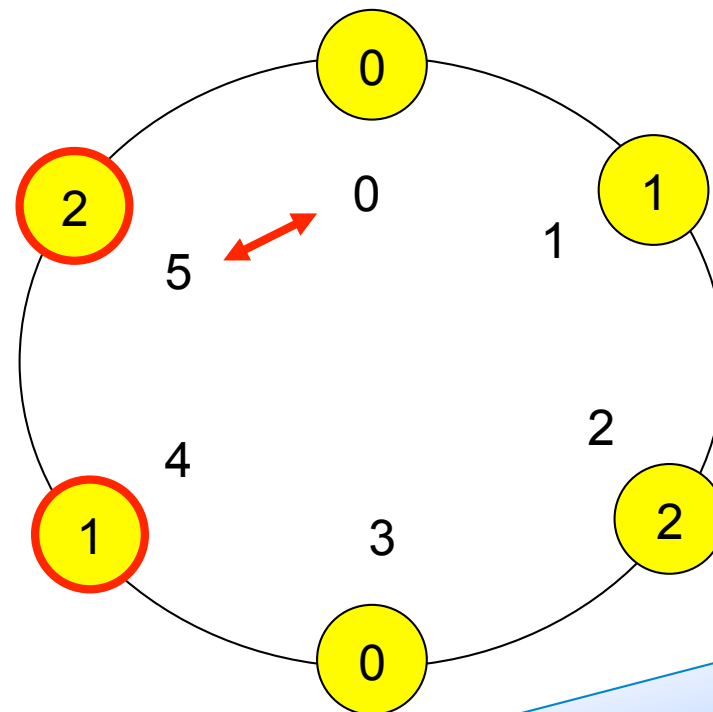
$$\delta_0 \xrightarrow{D_0} \delta_1 \xrightarrow{D_1} \delta_2 \xrightarrow{D_2} \dots$$

$$\begin{array}{c} \downarrow \quad \downarrow \quad \downarrow \\ \gamma_0 \xrightarrow{D_0} \gamma_1 \xrightarrow{D_1} \gamma_2 \xrightarrow{D_2} \dots \end{array}$$



After a fault or an arbitrary initialization

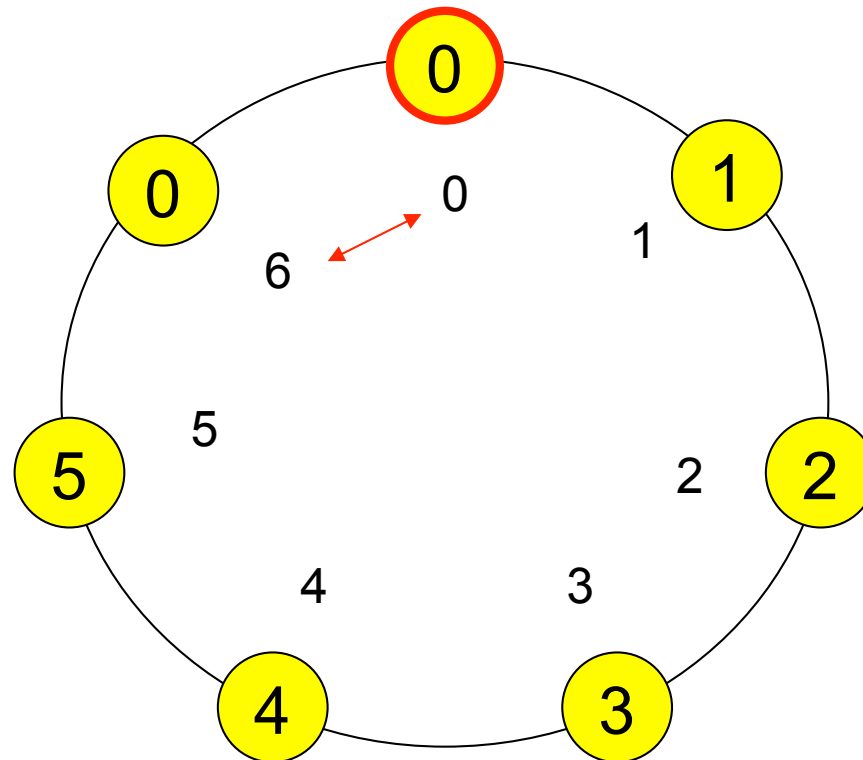
○ K=3

$$\delta_0 \xrightarrow{D_0} \delta_1 \xrightarrow{D_1} \delta_2 \xrightarrow{D_2} \dots$$
$$\downarrow \quad \downarrow \quad \downarrow$$
$$\gamma_0 \xrightarrow{D_0} \gamma_1 \xrightarrow{D_1} \gamma_2 \xrightarrow{D_2} \dots$$


DEADLOCK!

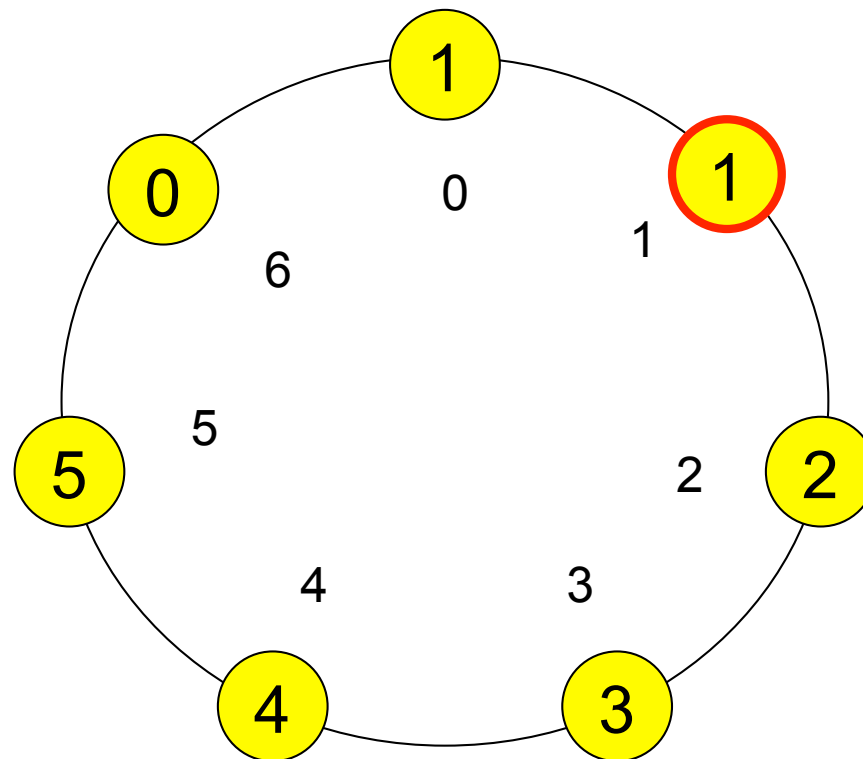
After a fault or an arbitrary initialization

○ K=6



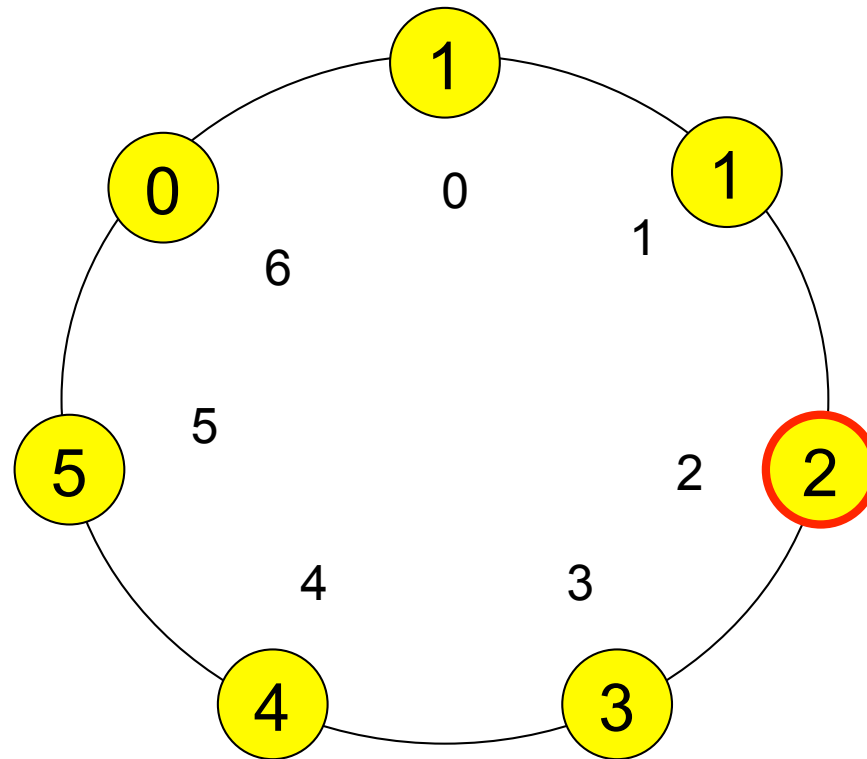
After a fault or an arbitrary initialization

○ K=6



After a fault or an arbitrary initialization

○ $K=6$



MUTUAL EXCLUSION!

[After a fault or an arbitrary initialization]

What are the conditions to be able to (re)-synchronize the system ?

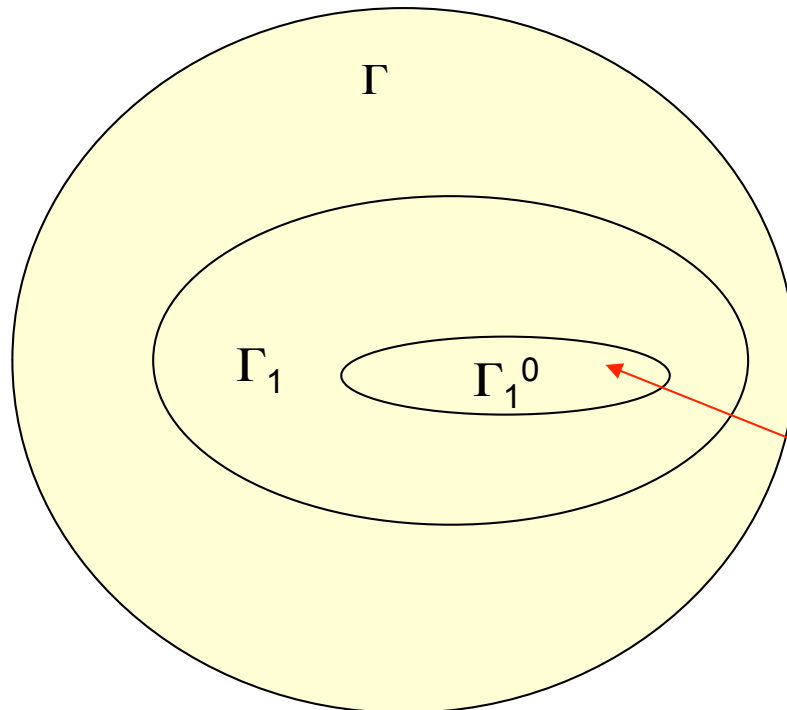
System Configurations

Γ : The register values are arbitrary

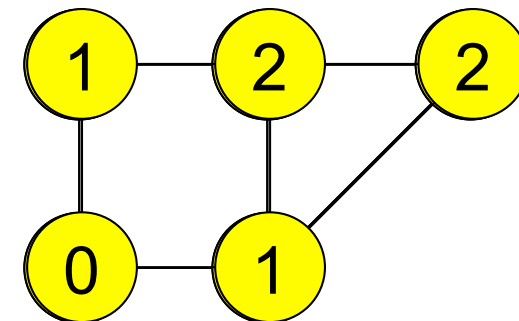
Γ_1 : The register of two neighboring process is less than or equal to 1

Γ_1^0 : There is no deadlock (there exists a compatible lifting to the configuration)

$K=4$



No Deadlock



System Configurations

Γ : The register values are arbitrary

Γ_1 : The register of two neighboring process is less than or equal to 1

Γ_1^0 : There is no deadlock (there exists a compatible lifting to the configuration)

THEOREM:

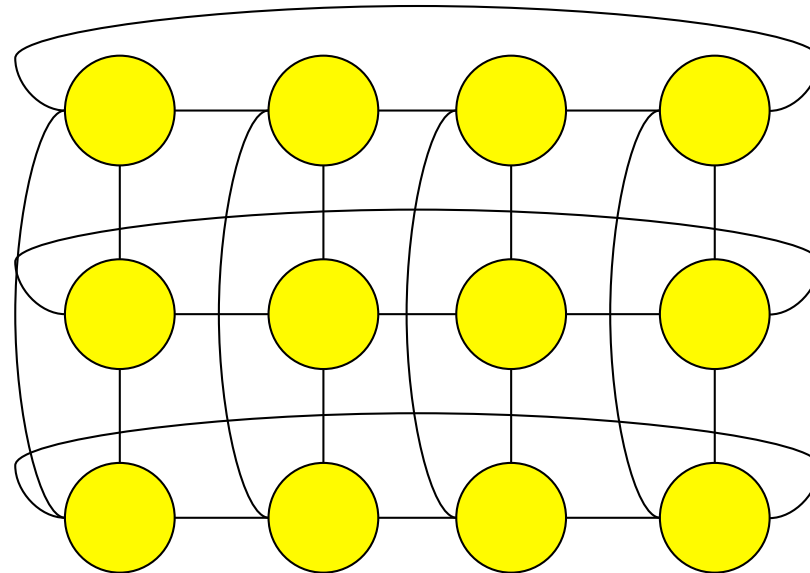
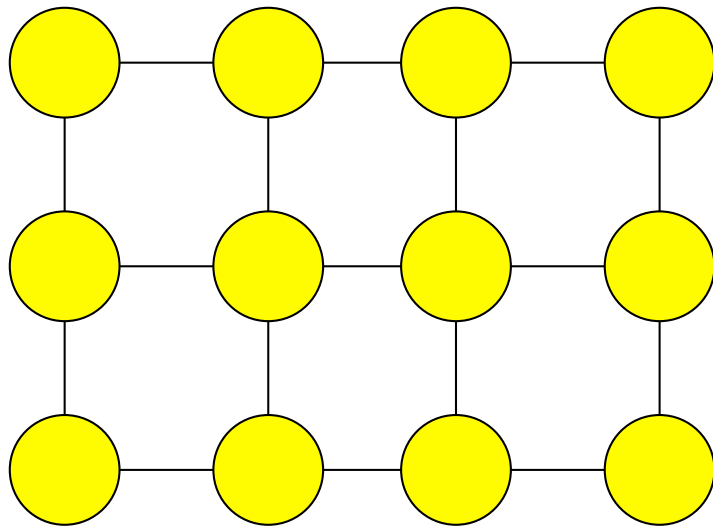
$$K > C_G \Rightarrow \Gamma_1 = \Gamma_1^0$$

- C_G ($C_G \leq n$), the *cyclomatic characteristic* of the graph:
Equal to the size of the greatest cycle in one of the cycle basis of G
where the size of the greatest cycle is minimum (equal to 2 if G is acyclic)

[Boulinier, Petit, and Villain, PODC 2004]

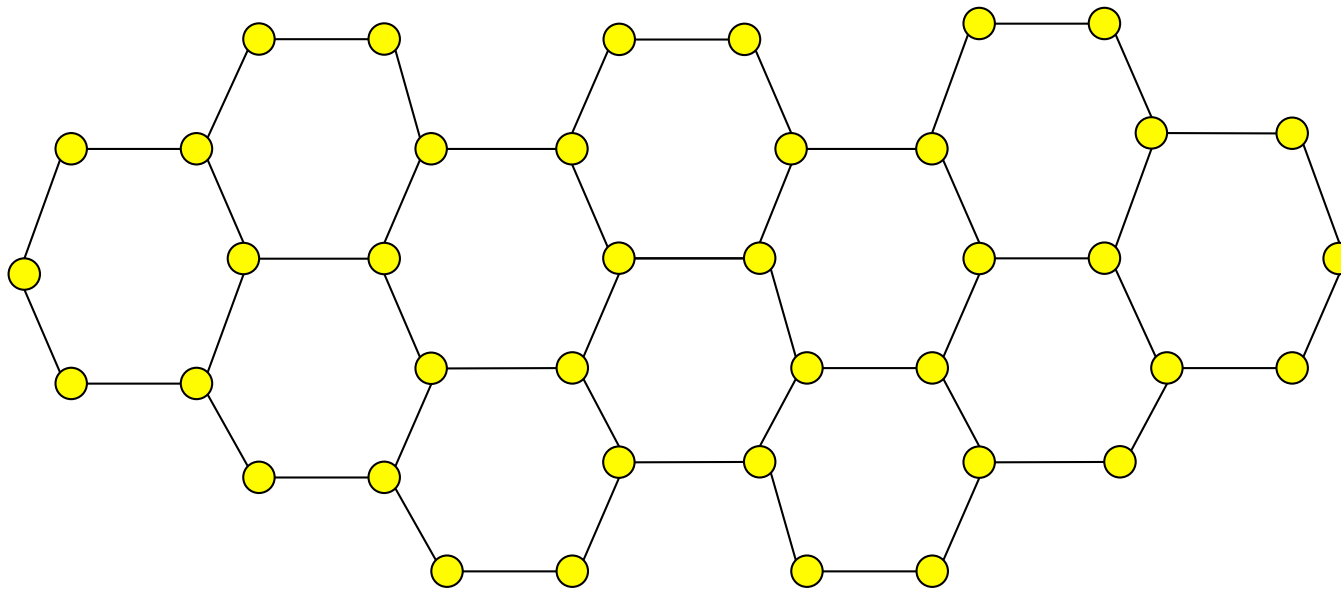
Cyclomatic characteristic of G

$C_G=4$ in meches and tories



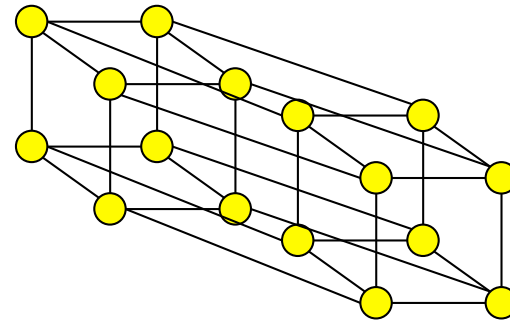
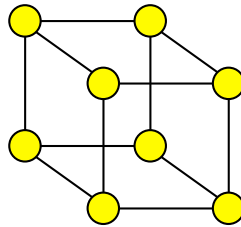
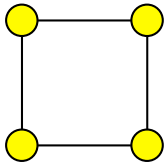
Cyclomatic characteristic of G

$C_G=6$ in honeycombs



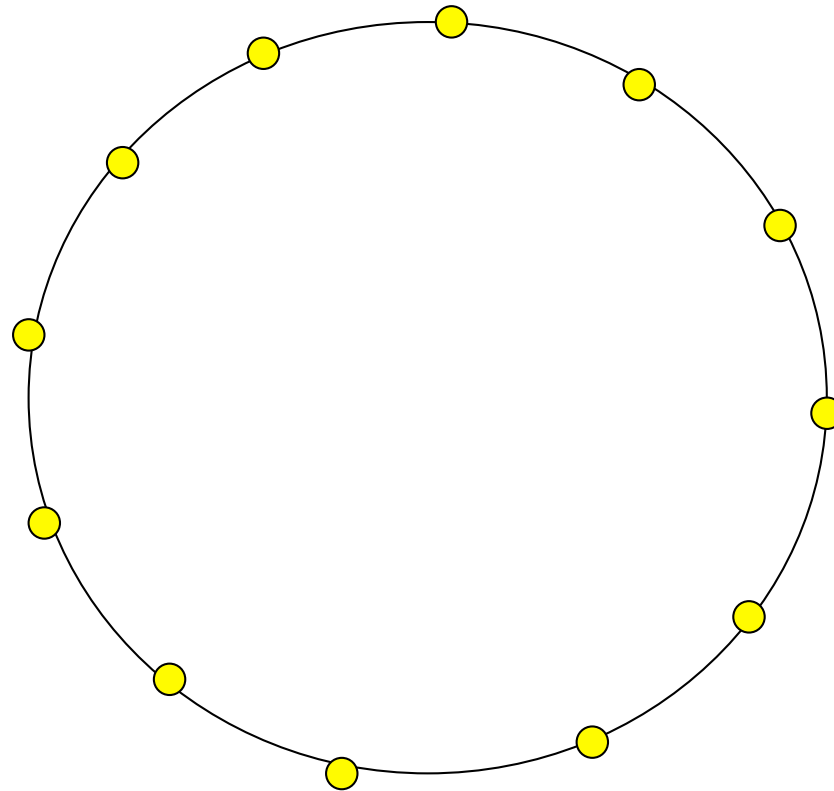
Cyclomatic characteristic of G

$C_G=4$ in hypercubes



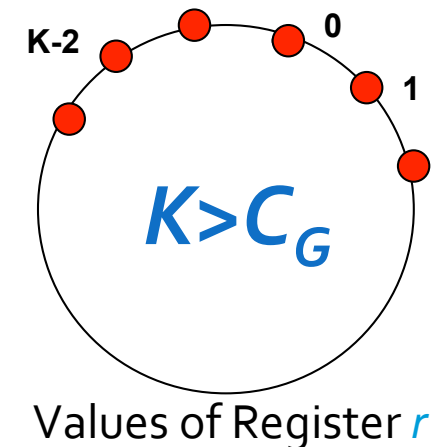
Cyclomatic characteristic of G

$C_G = n$ on rings



Logical Clock Synchronization

- To avoid deadlock due arbitrary initial values, K must be greater than C_G ($C_G \leq n$), the *cyclomatic characteristic* of the graph

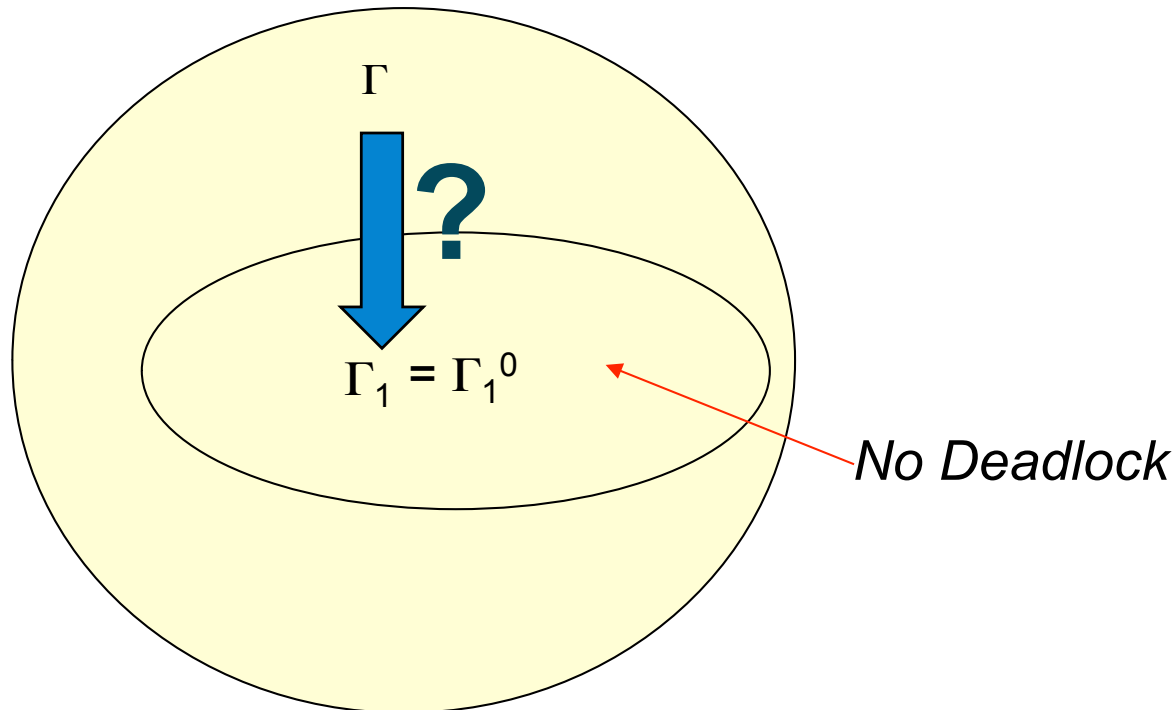


System Configurations

Γ : The register values are arbitrary

Γ_1 : The register of two neighboring process is less than or equal to 1

Γ_1^0 : There is no deadlock (there exists a compatible lifting to the configuration)



Stabilization

- Global Reset (Stabilizing PIF), implies a root or IDs
- Local Reset (also works in **anonymous** networks)

QUESTION:

*What is the motivation behind **anonymity**?*

"Only a few amount of bits allows to distinguish a huge number of nodes!"

Advantages of Anonymous Solutions

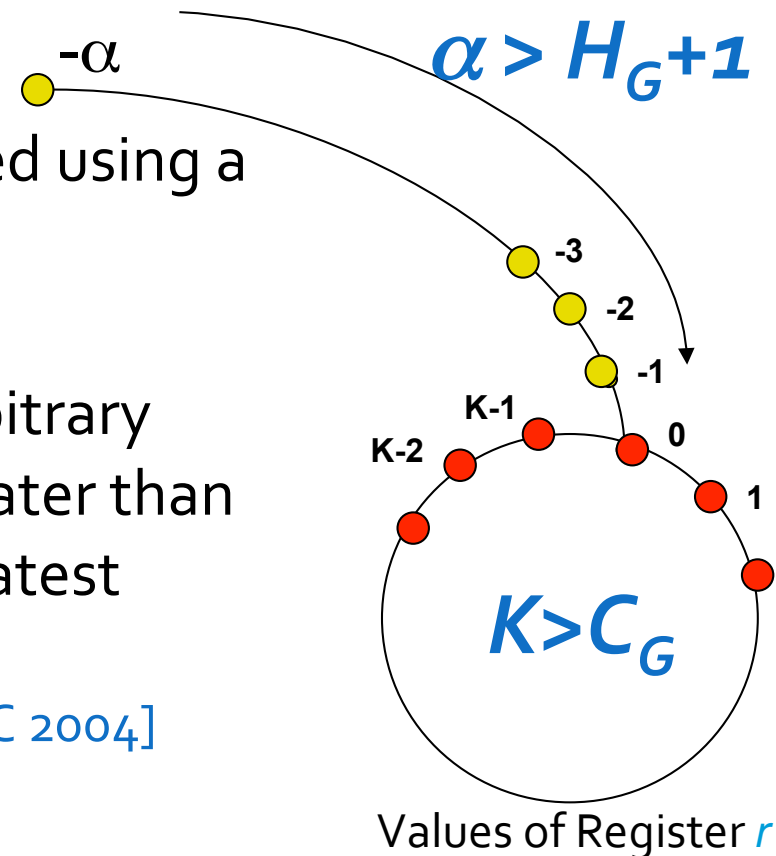
- Lack of (underlying) infrastructure
 - No unique identifier assignation or no central process
 - No maintenance of any distributed structure
- Economic advantages
- User privacy preserved
[Delporte-Gallet, Fauconnier, Guerraoui, and Ruppert, OPODIS 2007]
- No one-to-one routing
- Very suitable for sensor networks

Self-Stabilizing Logical Clock Synchronization

- To avoid starvations due arbitrary initial values, K must be greater than C_G ($C_G \leq n$), the *cyclomatic characteristic* of the graph

- Safety eventually guaranteed using a reset mechanism:
Register r is set in $[-\alpha, \dots, 0]$

- To avoid starvations due arbitrary initial values, α must be greater than $H_G + 1$ ($H_G \leq n$), the greatest chordless cycle of the graph
[Boulinier, Petit, and Villain, PODC 2004]



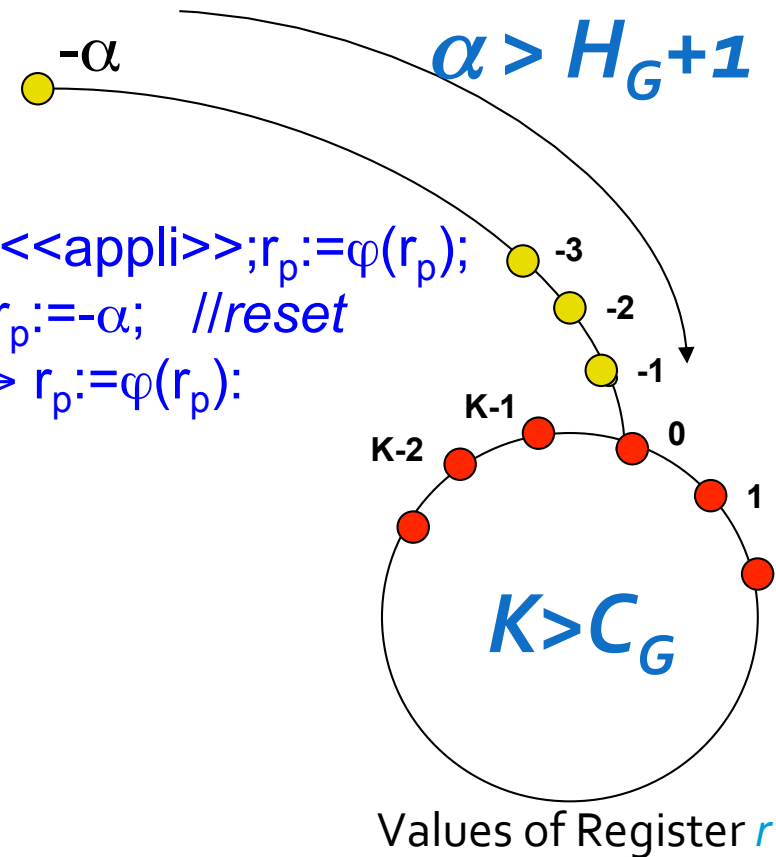
Self-Stabilizing Logical Clock Synchronization

SSSync

AN: $\forall q \in N_p: \text{Correct}_p(q) \wedge (\text{NormalStep}_p) \rightarrow \langle\langle \text{appli} \rangle\rangle; r_p := \varphi(r_p);$

AR: $\neg (\forall q \in N_p: \text{Correct}_p(q)) \wedge (r_p \notin \text{tail}\varphi) \rightarrow r_p := -\alpha; \text{ //reset}$

AC: $r_p \in \text{init}\varphi^* \wedge (\forall q \in N_p: r_q \in \text{init}\varphi^* \wedge r_p \leq \varphi(r_q)) \rightarrow r_p := \varphi(r_p);$



Asynchronous, Anonymous Logical Clock, Related Works

	# of states	Stabilizing Time
Gouda, Couvreur, Francez, 1992	$O(n^2)$	$O(nd)$
Dolev, 2000	$O(n^2)$	$O(d)$
○ SSSync(K, α, M)		
$\alpha=0, K > M \cdot C_G, M > H_G - 2$	$O(nd)$	$O(nd)$
$\alpha > H_G - 2, M=1, K > C_G$	$O(n)$	$O(n)$
Tree networks	$O(1)$	$O(d)$

Wave Algorithms in Anonymous Networks

- All existing solutions for coordination problems (either global, local or at distance ρ) are based on:
 - Existence of unique IDs (or a root)
 - Underlying wave mechanism

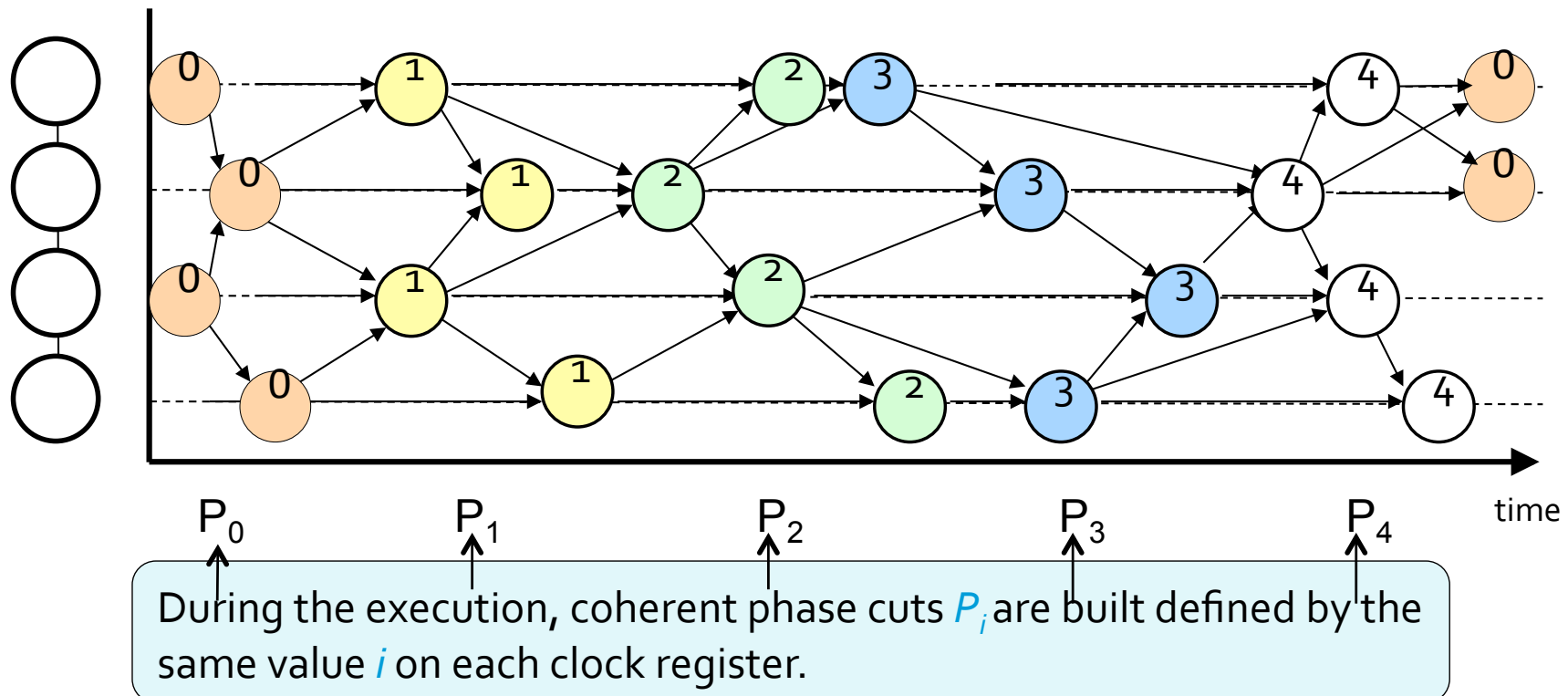
QUESTION:

*Can we provide wave algorithms using neither IDS nor root, i.e., working in an **anonymous** system?*

- Anonymous global ($\rho = \text{Diam}$) waves for non fault-tolerant systems
[Tel, Concurrency'88]

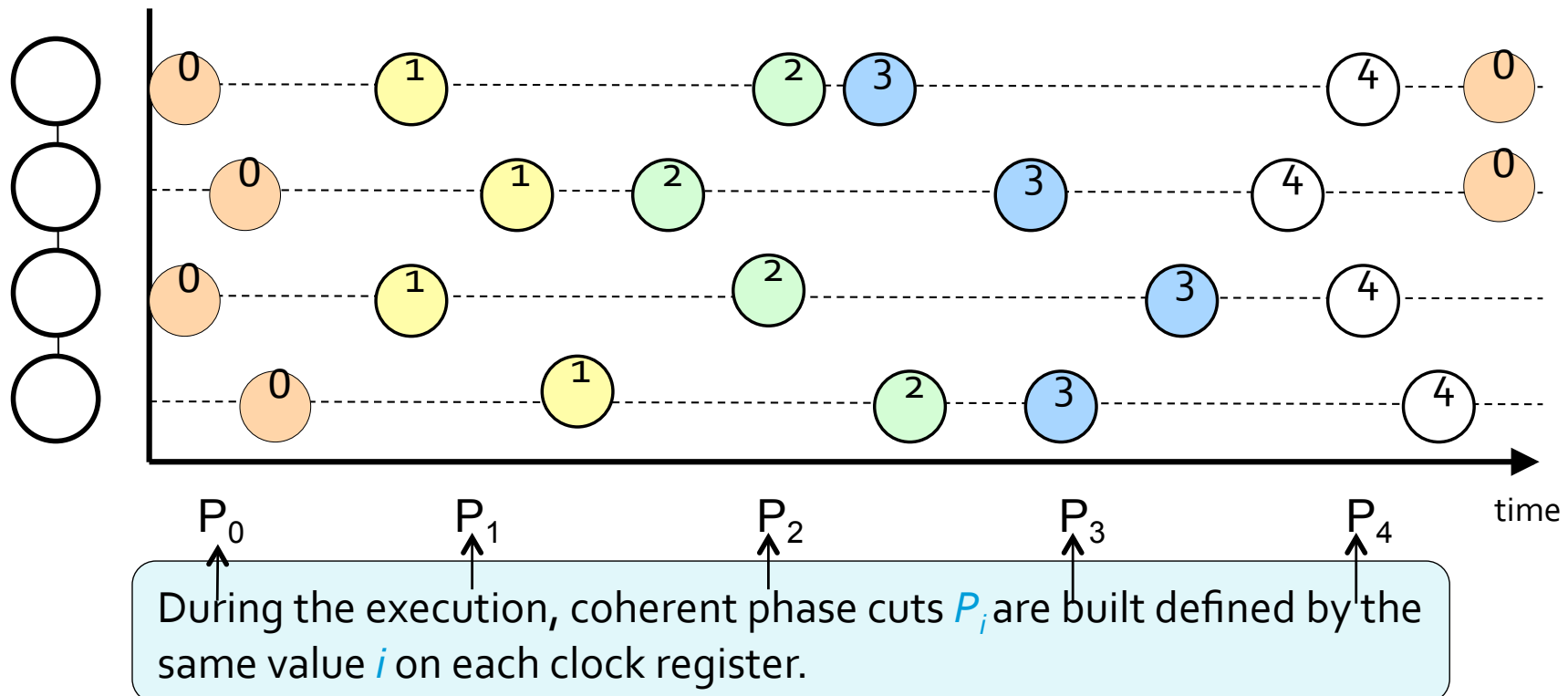
Logical Clock Synchronization in Asynchronous Settings

- Let us observe a possible execution of the self-stabilizing phase clock synchronization again
- The system (a chain of 4 nodes) is stabilized
- $K=5$



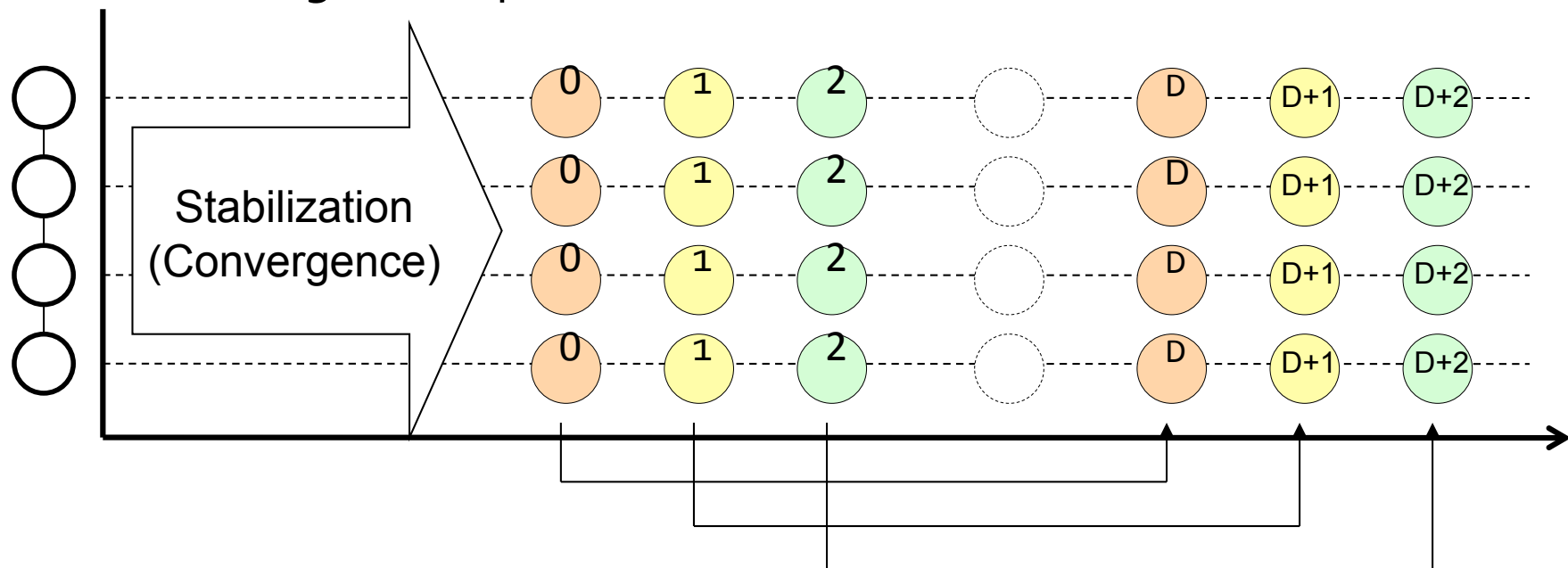
Logical Clock Synchronization in Asynchronous Settings

- Let us observe a possible execution of the self-stabilizing phase clock synchronization again
- The system (a chain of 4 nodes) is stabilized
- $K=5$



Phase Clock Synchronization in Asynchronous Settings

- Observe that:
 1. After stabilization, the phase clock synchronization algorithm provides a wave stream



2. In the future of a coherent phase P_i , the d^{th} phase clock incrementation causally depends on nodes at distance $\leq d$.

Self-Stabilizing ρ -Wavelets

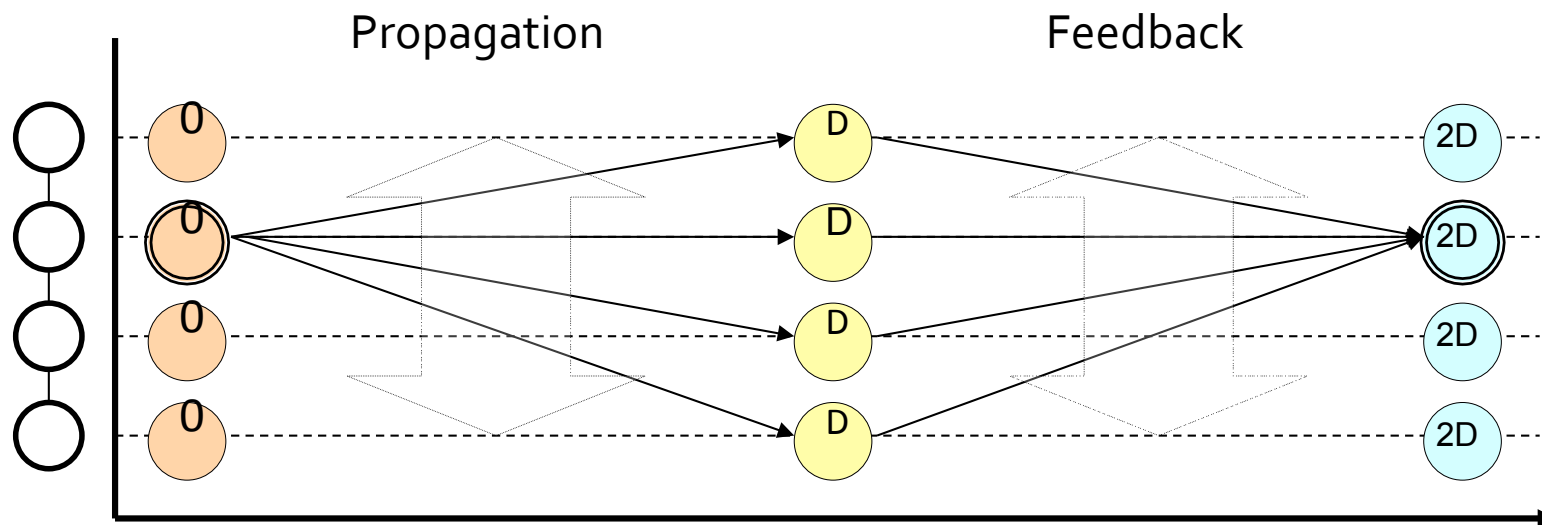
- When the clock register r is equal to a particular value, e.g., 0 , add a particular event “**DECIDE**”

THEOREM:

The self-stabilizing phase clock synchronization provides a self-stabilizing ρ -wavelet mechanism

Self-Stabilizing ρ -Wavelets

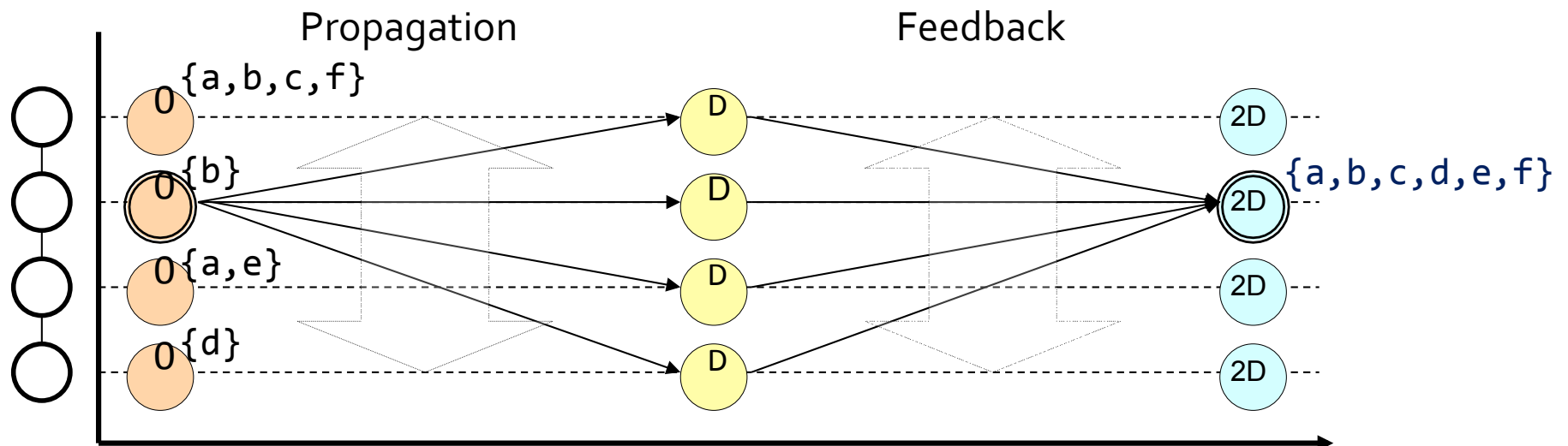
- For instance, if $\rho = \text{Diam}$, the above protocol provides a **Propagation of Information with Feedback** over an **anonymous** network



Self-Stabilizing ρ -Local Infimum Computation $\oplus$

- An infimum $\oplus$ over a set S is an **associative**, **commutative**, and **idempotent** ($x \oplus x = x$) binary operator.

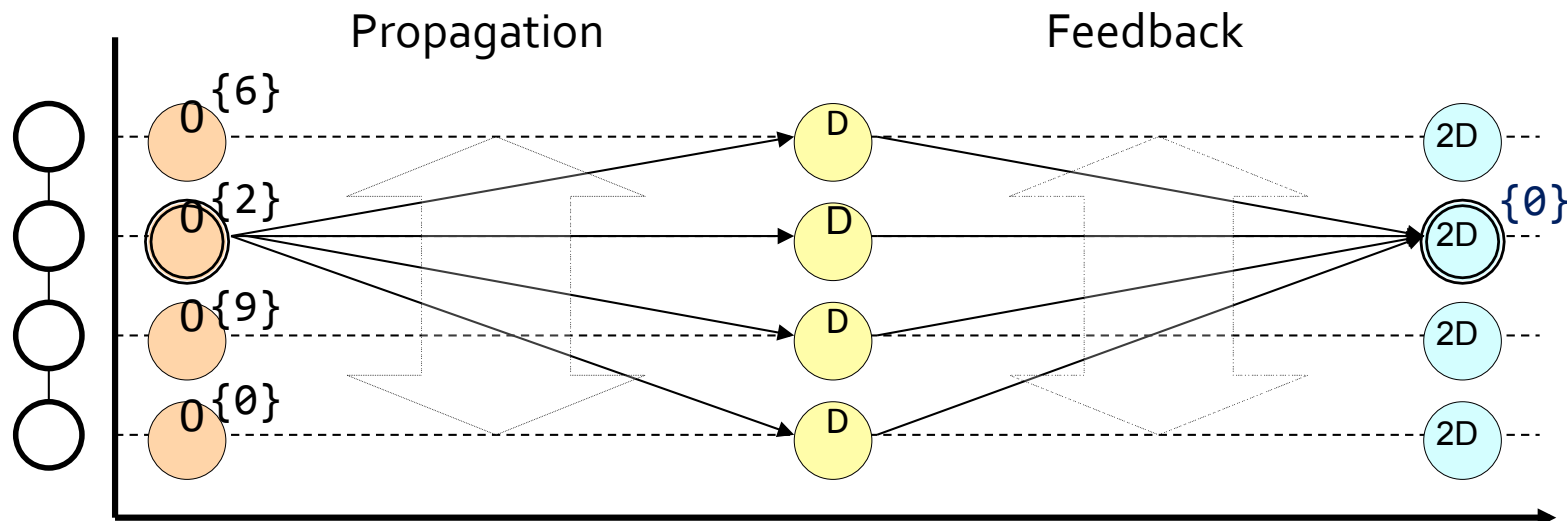
For instance, $\oplus = \cup$



Self-Stabilizing ρ -Local Infimum Computation $\oplus$

- An infimum $\oplus$ over a set S is an **associative**, **commutative**, and **idempotent** ($x \oplus x = x$) binary operator.

For instance, $\oplus = \min$

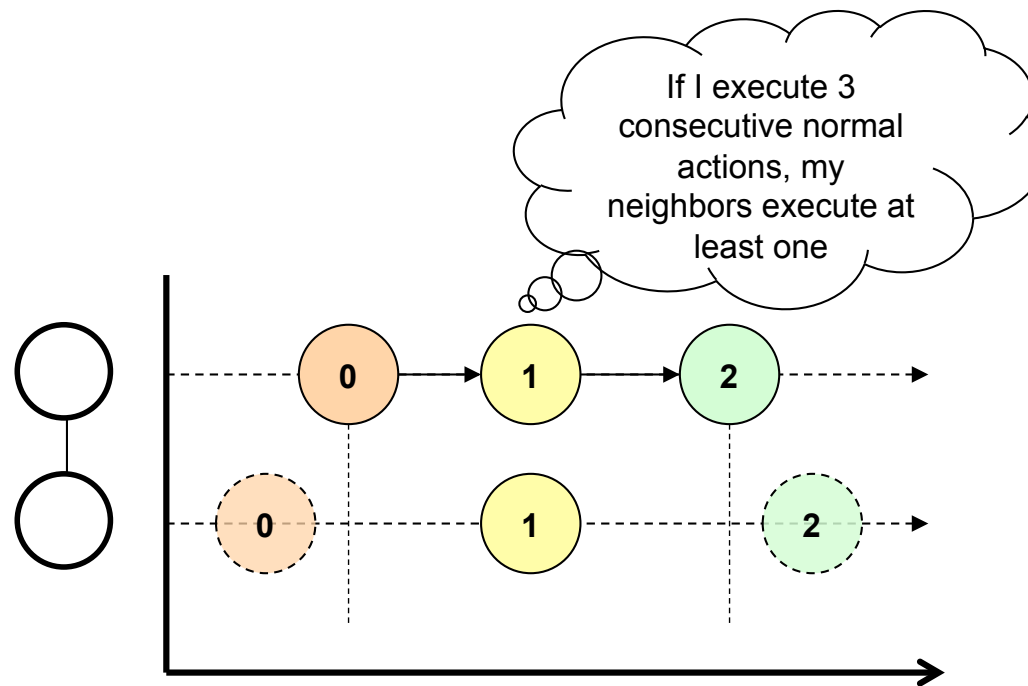


Enabling Snap-Stabilization in Anonymous Networks

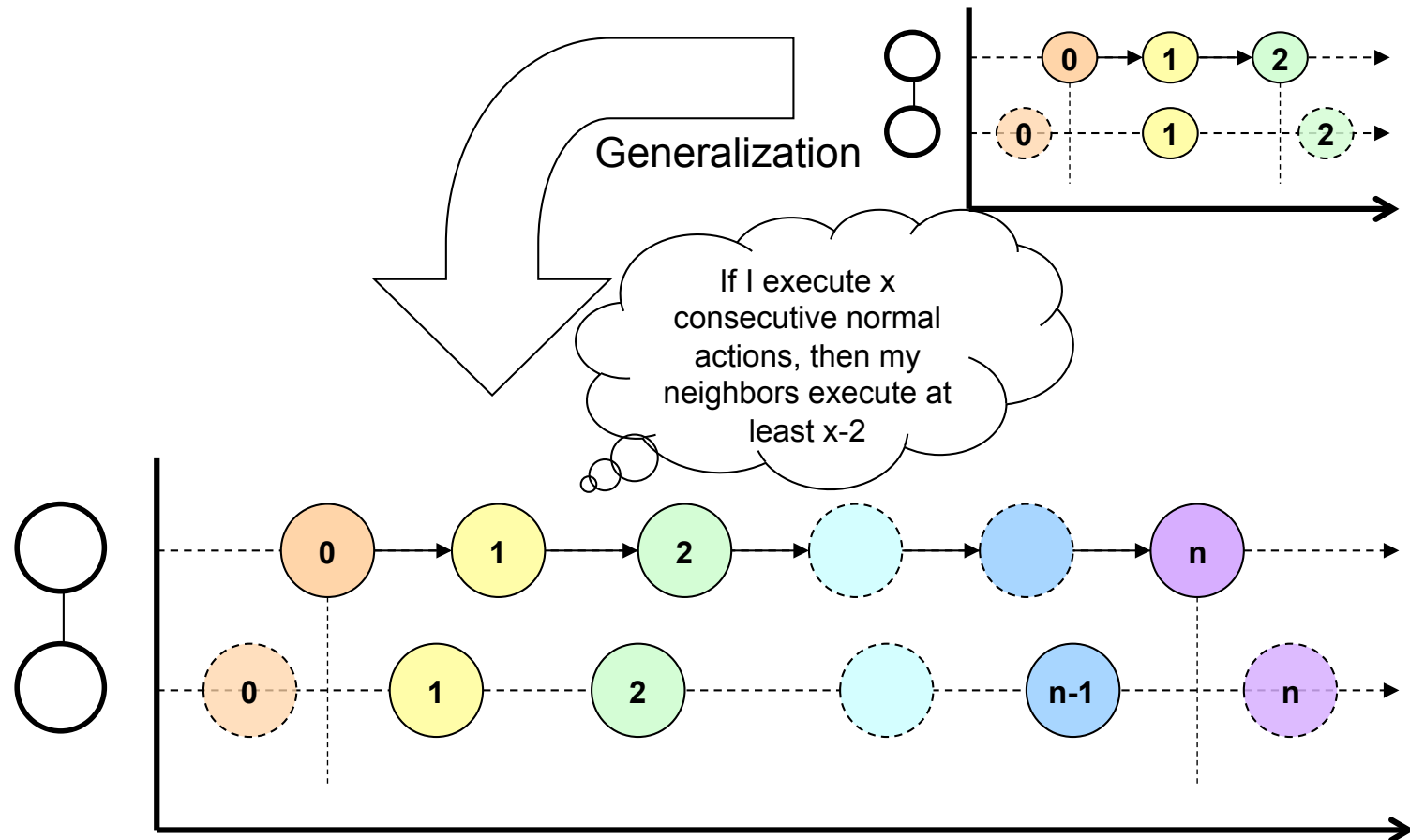
QUESTION:

Can we design snap-stabilizing waves algorithms in anonymous networks?

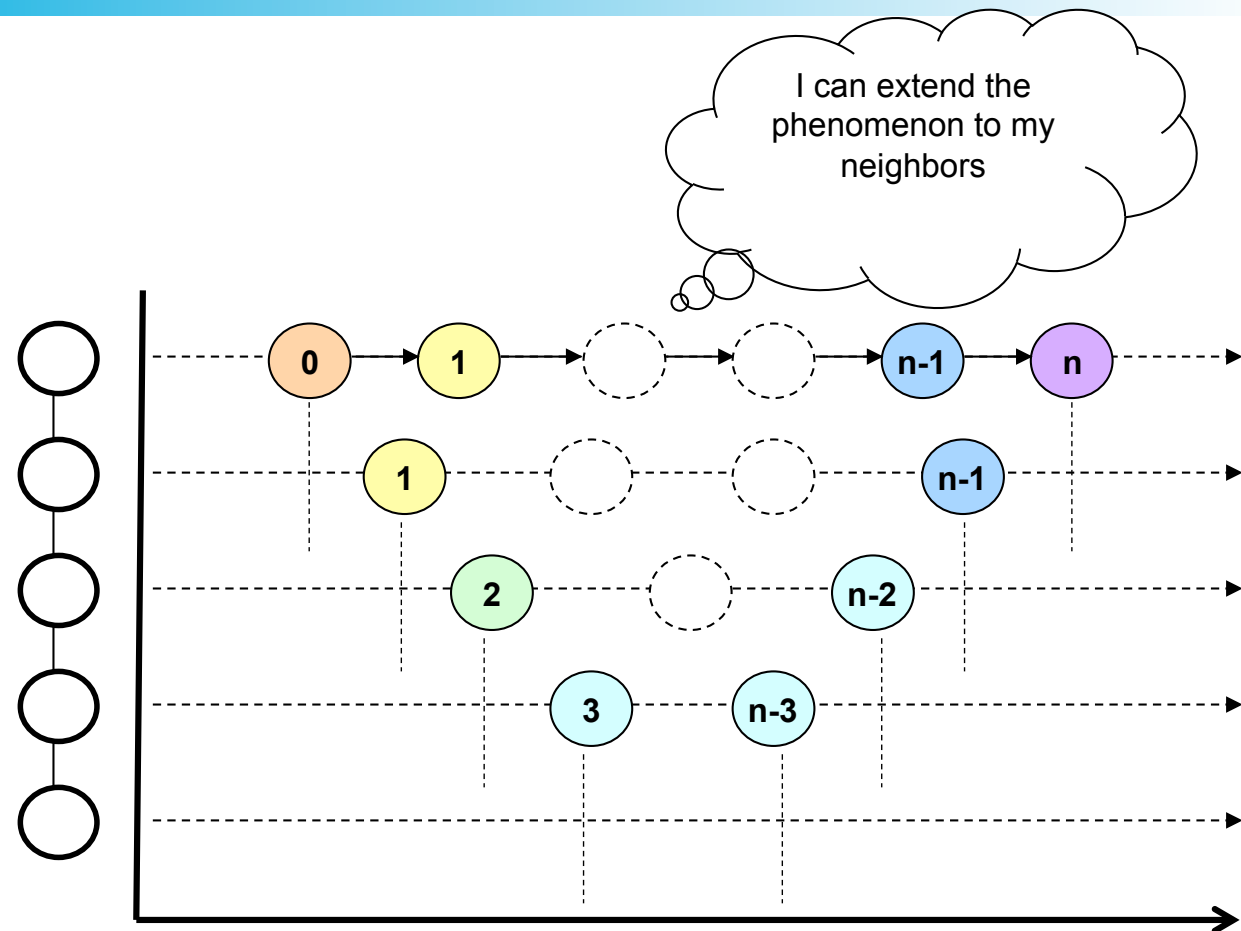
Coherent Cuts Detection



Coherent Cuts Detection

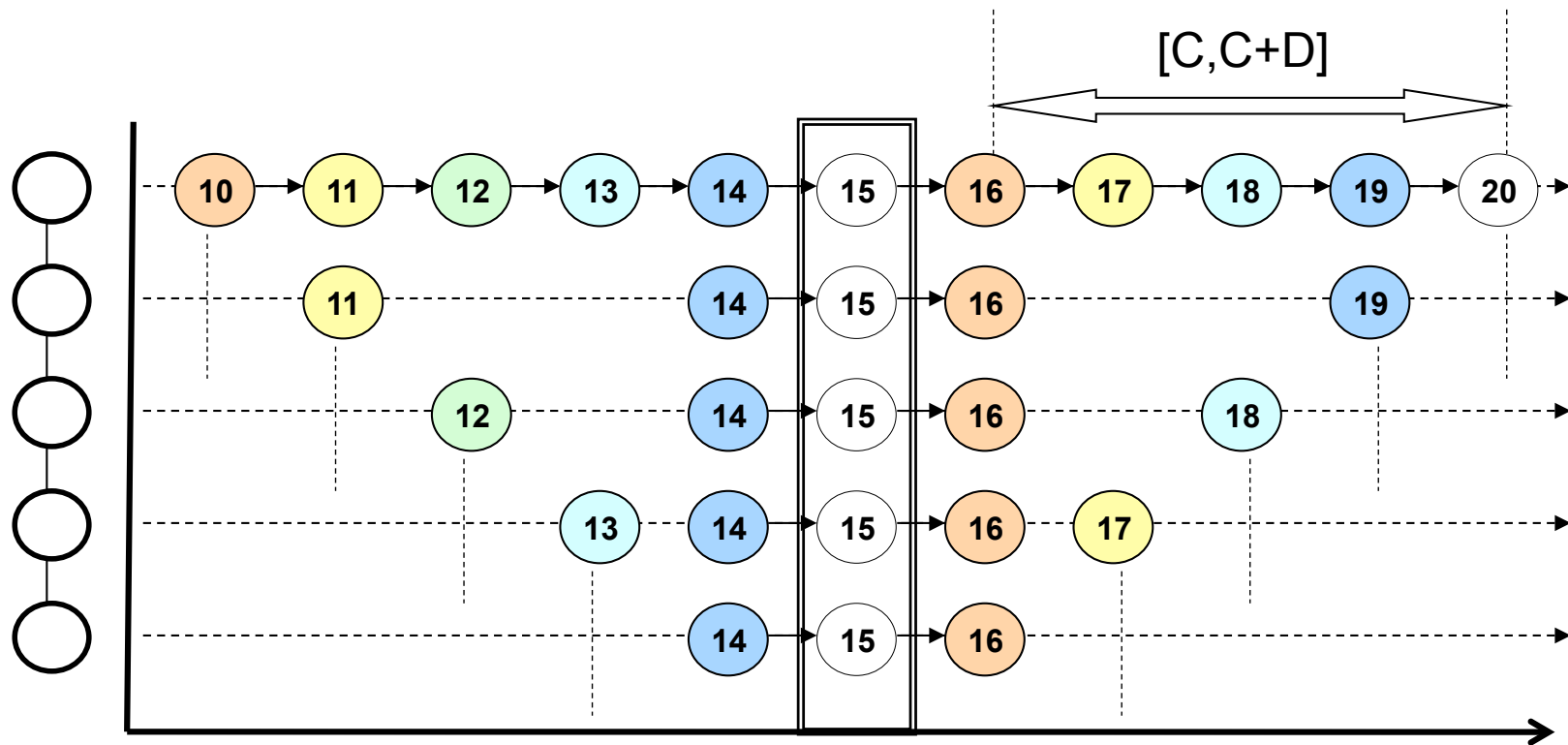


Coherent Cuts Detection

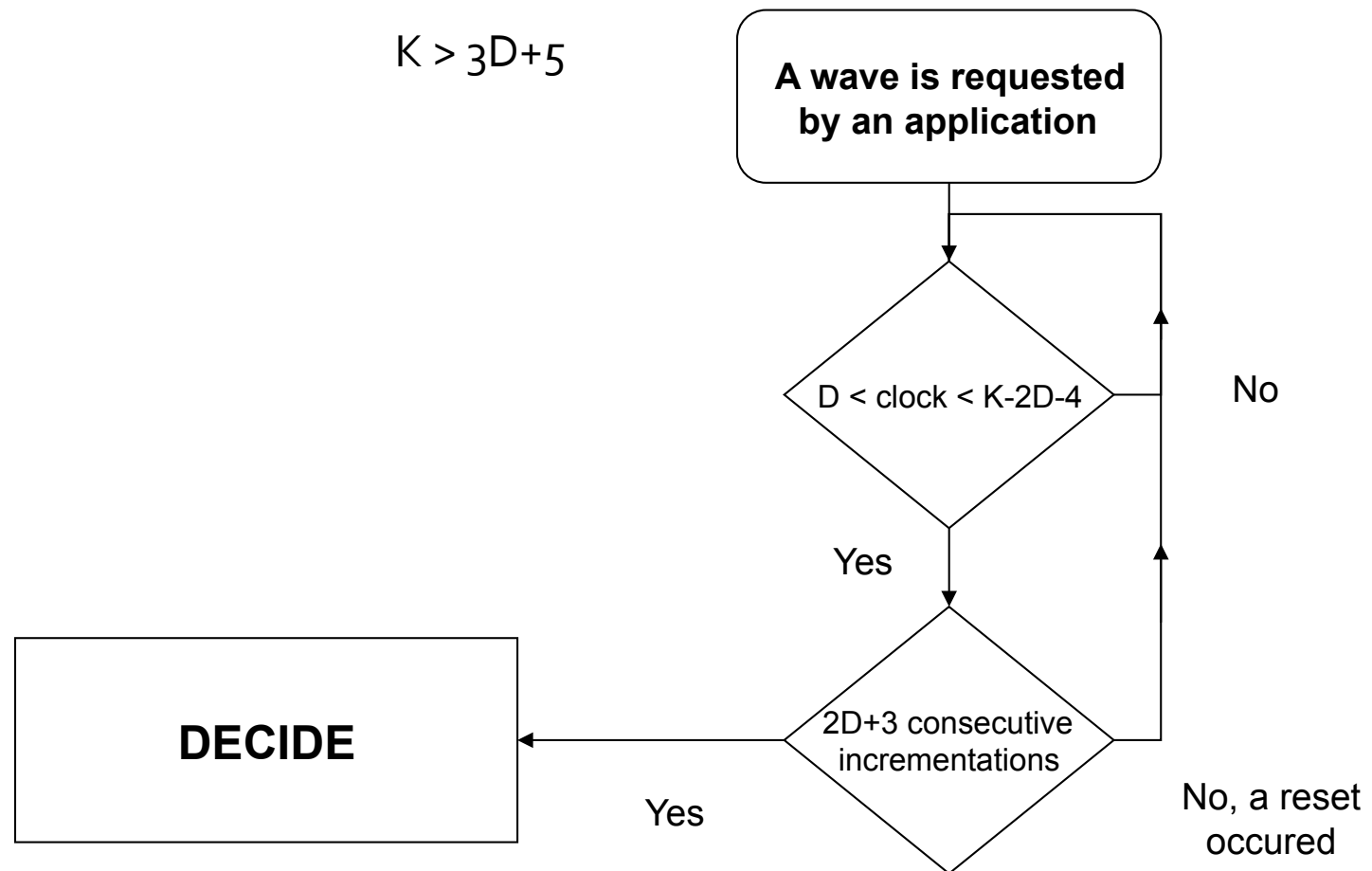


Coherent Cuts Detection

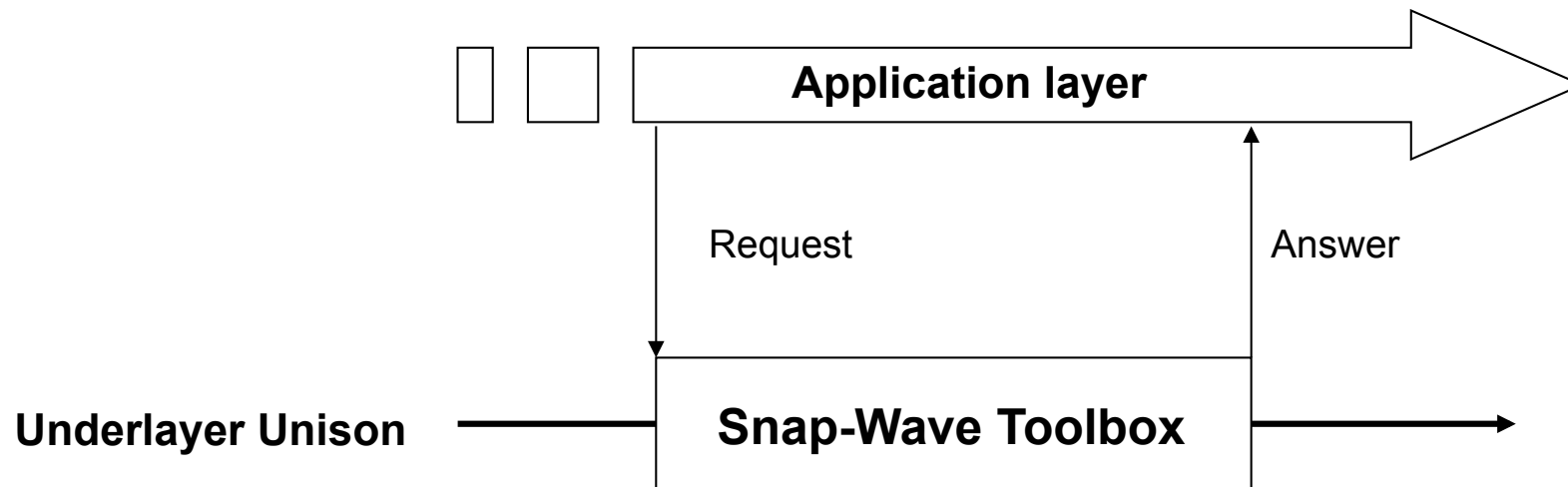
After an enough long normal action sequence (NA), a coherent phase is guaranteed in the past.



Snap-Stabilization Wave Protocol for Anonymous Networks



Enabling Snap-Stabilization in Anonymous Networks



Some references

- UNISON and its applications
 - Boulinier, Petit, Villain [PODC 04]
When Graph Theory helps Self-Stabilization
 - Boulinier, Petit, Villain, [Algorithmica 08]
Synchronous vs. Asynchronous Unison
 - Boulinier, Petit, [APDCM 08]
Self-Stabilizing Wavelets and ρ -Hop Coordination
 - Boulinier, Levert, Petit [ICDCN 08]
Snap-Stabilizing Waves in Anonymous Networks
- Snap-Stabilization
 - Bui, Datta, Petit, Villain [Distributed Computing 07]
Snap-stabilization and PIF in Tree Networks